

14.8.3 节练习

练习 14.44: 编写一个简单的桌面计算器使其能处理二元运算。



14.9 重载、类型转换与运算符

在 7.5.4 节（第 263 页）中我们看到由一个实参调用的非显式构造函数定义了一种隐式的类型转换，这种构造函数将实参类型的对象转换成类类型。我们同样能定义对于类类型的类型转换，通过定义类型转换运算符可以做到这一点。转换构造函数和类型转换运算符共同定义了类类型转换（class-type conversions），这样的转换有时也被称作用户定义的类型转换（user-defined conversions）。

14.9.1 类型转换运算符

类型转换运算符（conversion operator）是类的一种特殊成员函数，它负责将一个类类型的值转换成其他类型。类型转换函数的一般形式如下所示：

```
operator type() const;
```

其中 *type* 表示某种类型。类型转换运算符可以面向任意类型（除了 `void` 之外）进行定义，只要该类型能作为函数的返回类型（参见 6.1 节，第 184 页）。因此，我们不允许转换成数组或者函数类型，但允许转换成指针（包括数组指针及函数指针）或者引用类型。

类型转换运算符既没有显式的返回类型，也没有形参，而且必须定义成类的成员函数。类型转换运算符通常不应该改变待转换对象的内容，因此，类型转换运算符一般被定义成 `const` 成员。



一个类型转换函数必须是类的成员函数；它不能声明返回类型，形参列表也必须为空。类型转换函数通常应该是 `const`。

定义含有类型转换运算符的类

举个例子，我们定义一个比较简单的类，令其表示 0 到 255 之间的一个整数：

```
class SmallInt {
public:
    SmallInt(int i = 0): val(i)
    {
        if (i < 0 || i > 255)
            throw std::out_of_range("Bad SmallInt value");
    }
    operator int() const { return val; }
private:
    std::size_t val;
};
```

我们的 `SmallInt` 类既定义了向类类型的转换，也定义了从类类型向其他类型的转换。其中，构造函数将算术类型的值转换成 `SmallInt` 对象，而类型转换运算符将 `SmallInt` 对象转换成 `int`：

```
SmallInt si;
```

```

si = 4;          // 首先将 4 隐式地转换成 SmallInt, 然后调用 SmallInt::operator=
si + 3;          // 首先将 si 隐式地转换成 int, 然后执行整数的加法

```

尽管编译器一次只能执行一个用户定义的类型转换（参见 4.11.2 节，第 144 页），但是隐式的用户定义类型转换可以置于一个标准（内置）类型转换之前或之后（参见 4.11.1 节，第 141 页），并与其一起使用。因此，我们可以将任何算术类型传递给 SmallInt 的构造函数。类似的，我们也能使用类型转换运算符将一个 SmallInt 对象转换成 int，然后再将所得的 int 转换成任何其他算术类型：

```

// 内置类型转换将 double 实参转换成 int
SmallInt si = 3.14;          // 调用 SmallInt(int) 构造函数
// SmallInt 的类型转换运算符将 si 转换成 int
si + 3.14;                   // 内置类型转换将所得的 int 继续转换成 double

```

因为类型转换运算符是隐式执行的，所以无法给这些函数传递实参，当然也就不能在类型转换运算符的定义中使用任何形参。同时，尽管类型转换函数不负责指定返回类型，但实际上每个类型转换函数都会返回一个对应类型的值：

```

class SmallInt;
operator int(SmallInt&);          // 错误：不是成员函数
class SmallInt {
public:
    int operator int() const;      // 错误：指定了返回类型
    operator int(int = 0) const;   // 错误：参数列表不为空
    operator int*() const { return 42; } // 错误：42 不是一个指针
};

```

提示：避免过度使用类型转换函数

和使用重载运算符的经验一样，明智地使用类型转换运算符也能极大地简化类设计者的工作，同时使得使用类更加容易。然而，如果在类类型和转换类型之间不存在明显的映射关系，则这样的类型转换可能具有误导性。

例如，假设某个类表示 Date，我们也许会为它添加一个从 Date 到 int 的转换。然而，类型转换函数的返回值应该是什么？一种可能的解释是，函数返回一个十进制数，依次表示年、月、日，例如，July 30, 1989 可能转换为 int 值 19890730。同时还存在另外一种合理的解释，即类型转换运算符返回的 int 表示的是从某个时间节点（比如 January 1, 1970）开始经过的天数。显然这两种理解都合情合理，毕竟从形式上看它们产生的效果都是越靠后的日期对应的整数值越大，而且两种转换都有实际的用处。

问题在于 Date 类型的对象和 int 类型的值之间不存在明确的一对一映射关系。因此在此例中，不定义该类型转换运算符也许会更好。作为替代的手段，类可以定义一个或多个普通的成员函数以从各种不同形式中提取所需的信息。

类型转换运算符可能产生意外结果

在实践中，类很少提供类型转换运算符。在大多数情况下，如果类型转换自动发生，用户可能会感觉比较意外，而不是感觉受到了帮助。然而这条经验法则则存在一种例外情况：对于类来说，定义向 bool 的类型转换还是比较普遍的现象。

在 C++ 标准的早期版本中，如果类想定义一个向 bool 的类型转换，则它常常遇到一个问题：因为 bool 是一种算术类型，所以类类型的对象转换成 bool 后就能被用在任何

581

582

需要算术类型的上下文中。这样的类型转换可能引发意想不到的结果，特别是当 `istream` 含有向 `bool` 的类型转换时，下面的代码仍将编译通过：

```
int i = 42;
cin << i; // 如果向 bool 的类型转换不是显式的，则该代码在编译器看来将是合法的！
```

这段程序试图将输出运算符作用于输入流。因为 `istream` 本身并没有定义 `<<`，所以本来代码应该产生错误。然而，该代码能使用 `istream` 的 `bool` 类型转换运算符将 `cin` 转换成 `bool`，而这个 `bool` 值接着会被提升成 `int` 并用作内置的左移运算符的左侧运算对象。这样一来，提升后的 `bool` 值（1 或 0）最终会被左移 42 个位置。这一结果显然与我们的预期大相径庭。

显式的类型转换运算符

C++
11

为了防止这样的异常情况发生，C++11 新标准引入了显式的类型转换运算符（explicit conversion operator）：

```
class SmallInt {
public:
    // 编译器不会自动执行这一类型转换
    explicit operator int() const { return val; }
    // 其他成员与之前的版本一致
};
```

和显式的构造函数（参见 7.5.4 节，第 265 页）一样，编译器（通常）也不会将一个显式的类型转换运算符用于隐式类型转换：

```
SmallInt si = 3; // 正确：SmallInt 的构造函数不是显式的
si + 3;           // 错误：此处需要隐式的类型转换，但类的运算符是显式的
static_cast<int>(si) + 3; // 正确：显式地请求类型转换
```

当类型转换运算符是显式的时，我们也能执行类型转换，不过必须通过显式的强制类型转换才可以。

该规定存在一个例外，即如果表达式被用作条件，则编译器会将显式的类型转换自动应用于它。换句话说，当表达式出现在下列位置时，显式的类型转换将被隐式地执行：

- `if`、`while` 及 `do` 语句的条件部分
- `for` 语句头的条件表达式
- 逻辑非运算符（`!`）、逻辑或运算符（`||`）、逻辑与运算符（`&&`）的运算对象
- 条件运算符（`? :`）的条件表达式。

583

转换为 `bool`

在标准库的早期版本中，`IO` 类型定义了向 `void*` 的转换规则，以求避免上面提到的问题。在 C++11 新标准下，`IO` 标准库通过定义一个向 `bool` 的显式类型转换实现同样的目的。

无论我们什么时候在条件中使用流对象，都会使用为 `IO` 类型定义的 `operator bool`。例如：

```
while (std::cin >> value)
```

`while` 语句的条件执行输入运算符，它负责将数据读入到 `value` 并返回 `cin`。为了对条件求值，`cin` 被 `istream` `operator bool` 类型转换函数隐式地执行了转换。如果 `cin` 的条件状态是 `good`（参见 8.1.2 节，第 280 页），则该函数返回为真；否则该函数返回为假。



向 `bool` 的类型转换通常用在条件部分，因此 `operator bool` 一般定义成 `explicit` 的。

14.9.1 节练习

练习 14.45: 编写类型转换运算符将一个 `Sales_data` 对象分别转换成 `string` 和 `double`，你认为这些运算符的返回值应该是什么？

练习 14.46: 你认为应该为 `Sales_data` 类定义上面两种类型转换运算符吗？应该把它们声明成 `explicit` 的吗？为什么？

练习 14.47: 说明下面这两个类型转换运算符的区别。

```
struct Integral {
    operator const int();
    operator int() const;
};
```

练习 14.48: 你在 7.5.1 节的练习 7.40（第 261 页）中曾经选择并编写了一个类，你认为它应该含有向 `bool` 的类型转换运算符吗？如果是，解释原因并说明该运算符是否应该是 `explicit` 的；如果不是，也请解释原因。

练习 14.49: 为上一题提到的类定义一个转换目标是 `bool` 的类型转换运算符，先不用在意这么做是否应该。

14.9.2 避免有歧义性的类型转换



如果类中包含一个或多个类型转换，则必须确保在类类型和目标类型之间只存在唯一一种转换方式。否则的话，我们编写的代码将很可能会具有歧义性。

在两种情况下可能产生多重转换路径。第一种情况是两个类提供相同的类型转换：例如，当 `A` 类定义了一个接受 `B` 类对象的转换构造函数，同时 `B` 类定义了一个转换目标是 `A` 类的类型转换运算符时，我们就说它们提供了相同的类型转换。

第二种情况是类定义了多个转换规则，而这些转换涉及的类型本身可以通过其他类型转换联系在一起。最典型的例子是算术运算符，对某个给定的类来说，最好只定义最多一个与算术类型有关的转换规则。

◀ 584



通常情况下，不要为类定义相同的类型转换，也不要类中定义两个及两个以上转换源或转换目标是算术类型的转换。

实参匹配和相同的类型转换

在下面的例子中，我们定义了两种将 `B` 转换成 `A` 的方法：一种使用 `B` 的类型转换运算符、另一种使用 `A` 的以 `B` 为参数的构造函数：

```
// 最好不要在两个类之间构建相同的类型转换
struct B;
struct A {
    A() = default;
    A(const B&);           // 把一个 B 转换成 A
    // 其他数据成员
```

```
};
struct B {
    operator A() const; // 也是把一个 B 转换成 A
    // 其他数据成员
};
A f(const A&);
B b;
A a = f(b); // 二义性错误: 含义是 f(B::operator A())
            // 还是 f(A::A(const B&))?
```

因为同时存在两种由 B 获得 A 的方法, 所以造成编译器无法判断应该运行哪个类型转换, 也就是说, 对 f 的调用存在二义性。该调用可以使用以 B 为参数的 A 的构造函数, 也可以使用 B 当中把 B 转换成 A 的类型转换运算符。因为这两个函数效果相当、难分伯仲, 所以该调用将产生错误。

如果我们确实想执行上述的调用, 就不得不显式地调用类型转换运算符或者转换构造函数:

```
A a1 = f(b.operator A()); // 正确: 使用 B 的类型转换运算符
A a2 = f(A(b));           // 正确: 使用 A 的构造函数
```

值得注意的是, 我们无法使用强制类型转换来解决二义性问题, 因为强制类型转换本身也面临二义性。

二义性与转换目标为内置类型的多重类型转换

另外如果类定义了一组类型转换, 它们的转换源 (或者转换目标) 类型本身可以通过其他类型转换联系在一起, 则同样会产生二义性的问题。最简单也是最困扰我们的例子就是类当中定义了多个参数都是算术类型的构造函数, 或者转换目标都是算术类型的类型转换运算符。

例如, 在下面的类中包含两个转换构造函数, 它们的参数是两种不同的算术类型; 同时还包含两个类型转换运算符, 它们的转换目标也恰好是两种不同的算术类型:

585

```
struct A {
    A(int = 0);           // 最好不要创建两个转换源都是算术类型的类型转换
    A(double);
    operator int() const; // 最好不要创建两个转换对象都是算术类型的类型转换
    operator double() const;
    // 其他成员
};
void f2(long double);
A a;
f2(a); // 二义性错误: 含义是 f(A::operator int())
        // 还是 f(A::operator double())?

long lg;
A a2(lg); // 二义性错误: 含义是 A::A(int) 还是 A::A(double)?
```

在对 f2 的调用中, 哪个类型转换都无法精确匹配 long double。然而这两个类型转换都可以使用, 只要后面再执行一次生成 long double 的标准类型转换即可。因此, 在上面的两个类型转换中哪个都不比另一个更好, 调用将产生二义性。

当我们试图用 `long` 初始化 `a2` 时也遇到了同样问题，哪个构造函数都无法精确匹配 `long` 类型。它们在使用构造函数前都要求先将实参进行类型转换：

- 先执行 `long` 到 `double` 的标准类型转换，再执行 `A(double)`
- 先执行 `long` 到 `int` 的标准类型转换，再执行 `A(int)`

编译器没办法区分这两种转换序列的好坏，因此该调用将产生二义性。

调用 `f2` 及初始化 `a2` 的过程之所以会产生二义性，根本原因是它们所需的标准类型转换级别一致（参见 6.6.1 节，第 219 页）。当我们使用用户定义的类型转换时，如果转换过程包含标准类型转换，则标准类型转换的级别将决定编译器选择最佳匹配的过程：

```
short s = 42;
// 把 short 提升成 int 优于把 short 转换成 double
A a3(s);                // 使用 A::A(int)
```

在此例中，把 `short` 提升成 `int` 的操作要优于把 `short` 转换成 `double` 的操作，因此编译器将使用 `A::A(int)` 构造函数构造 `a3`，其中实参是 `s`（提升后）的值。



当我们使用两个用户定义的类型转换时，如果转换函数之前或之后存在标准类型转换，则标准类型转换将决定最佳匹配到底是哪个。

提示：类型转换与运算符

586

要想正确地设计类的重载运算符、转换构造函数及类型转换函数，必须加倍小心。尤其是当类同时定义了类型转换运算符及重载运算符时特别容易产生二义性。以下的经验规则可能对你有所帮助：

- 不要令两个类执行相同的类型转换：如果 `Foo` 类有一个接受 `Bar` 类对象的构造函数，则不要在 `Bar` 类中再定义转换目标是 `Foo` 类的类型转换运算符。
- 避免转换目标是内置算术类型的类型转换。特别是当你已经定义了一个转换成算术类型的类型转换时，接下来
 - 不要再定义接受算术类型的重载运算符。如果用户需要使用这样的运算符，则类型转换操作将转换你的类型的对象，然后使用内置的运算符。
 - 不要定义转换到多种算术类型的类型转换。让标准类型转换完成向其他算术类型转换的工作。

一言以蔽之：除了显式地向 `bool` 类型的转换之外，我们应该尽量避免定义类型转换函数并尽可能地限制那些“显然正确”的非显式构造函数。

重载函数与转换构造函数

当我们调用重载的函数时，从多个类型转换中进行选择将变得更加复杂。如果两个或多个类型转换都提供了同一种可行匹配，则这些类型转换一样好。

举个例子，当几个重载函数的参数分属不同的类类型时，如果这些类恰好定义了同样的转换构造函数，则二义性问题将进一步提升：

```
struct C {
    C(int);
    // 其他成员
```

```
};
struct D {
    D(int);
    // 其他成员
};
void manip(const C&);
void manip(const D&);
manip(10);           // 二义性错误: 含义是 manip(C(10)) 还是 manip(D(10))
```

其中 C 和 D 都包含接受 int 的构造函数，两个构造函数各自匹配 manip 的一个版本。因此调用将具有二义性：它的含义可能是把 int 转换成 C，然后调用 manip 的第一个版本；也可能是把 int 转换成 D，然后调用 manip 的第二个版本。

调用者可以显式地构造正确的类型从而消除二义性：

```
manip(C(10));       // 正确: 调用 manip(const C&)
```



如果在调用重载函数时我们需要使用构造函数或者强制类型转换来改变实参的类型，则这通常意味着程序的设计存在不足。

重载函数与用户定义的类型转换

当调用重载函数时，如果两个（或多个）用户定义的类型转换都提供了可行匹配，则认为这些类型转换一样好。在这个过程中，我们不会考虑任何可能出现的标准类型转换的级别。只有当重载函数能通过同一个类型转换函数得到匹配时，我们才会考虑其中出现的标准类型转换。

例如当我们调用 manip 时，即使其中一个类定义了需要对实参进行标准类型转换的构造函数，这次调用仍然会具有二义性：

```
struct E {
    E(double);
    // 其他成员
};
void manip2(const C&);
void manip2(const E&);
// 二义性错误: 两个不同的用户定义的类型转换都能用在此处
manip2(10);           // 含义是 manip2(C(10)) 还是 manip2(E(double(10)))
```

在此例中，C 有一个转换源为 int 的类型转换，E 有一个转换源为 double 的类型转换。对于 manip2(10) 来说，两个 manip2 函数都是可行的：

- manip2(const C&) 是可行的，因为 C 有一个接受 int 的转换构造函数，该构造函数与实参精确匹配。
- manip2(const E&) 是可行的，因为 E 有一个接受 double 的转换构造函数，而且为了使用该函数我们可以利用标准类型转换把 int 转换成所需的类型。

因为调用重载函数所请求的用户定义的类型转换不止一个且彼此不同，所以该调用具有二义性。即使其中一个调用需要额外的标准类型转换而另一个调用能精确匹配，编译器也会将该调用标示为错误。



在调用重载函数时，如果需要额外的标准类型转换，则该转换的级别只有当所有可行函数都请求同一个用户定义的类型转换时才有用。如果所需的用户定义的类型转换不止一个，则该调用具有二义性。

14.9.2 节练习

练习 14.50: 在初始化 `ex1` 和 `ex2` 的过程中，可能用到哪些类类型的转换序列呢？说明初始化是否正确并解释原因。

```
struct LongDouble {
    LongDouble(double = 0.0);
    operator double();
    operator float();
};
LongDouble ldObj;
int ex1 = ldObj;
float ex2 = ldObj;
```

练习 14.51: 在调用 `calc` 的过程中，可能用到哪些类型转换序列呢？说明最佳可行函数是如何被选出来的。

```
void calc(int);
void calc(LongDouble);
double dval;
calc(dval); // 哪个 calc?
```

14.9.3 函数匹配与重载运算符



重载的运算符也是重载的函数。因此，通用的函数匹配规则（参见 6.4 节，第 208 页）同样适用于判断在给定的表达式中到底应该使用内置运算符还是重载的运算符。不过当运算符函数出现在表达式中时，候选函数集的规模要比我们使用调用运算符调用函数时更大。如果 `a` 是一种类类型，则表达式 `a sym b` 可能是

```
a.operatorsym(b); // a 有一个 operatorsym 成员函数
operatorsym(a, b); // operatorsym 是一个普通函数
```

和普通函数调用不同，我们不能通过调用的形式来区分当前调用的是成员函数还是非成员函数。

当我们使用重载运算符作用于类类型的运算对象时，候选函数中包含该运算符的普通非成员版本和内置版本。除此之外，如果左侧运算对象是类类型，则定义在该类中的运算符的重载版本也包含在候选函数内。

当我们调用一个命名的函数时，具有该名字的成员函数和非成员函数不会彼此重载，这是因为我们用来调用命名函数的语法形式对于成员函数和非成员函数来说是不相同的。当我们通过类类型的对象（或者该对象的指针及引用）进行函数调用时，只考虑该类的成员函数。而当我们在表达式中使用重载的运算符时，无法判断正在使用的是成员函数还是非成员函数，因此二者都应该在考虑的范围内。



表达式中运算符的候选函数集既应该包括成员函数，也应该包括非成员函数。

举个例子，我们为 SmallInt 类定义一个加法运算符：

```
class SmallInt {
    friend
    SmallInt operator+(const SmallInt&, const SmallInt&);
public:
    SmallInt(int = 0);                // 转换源为 int 的类型转换
    operator int() const { return val; } // 转换目标为 int 的类型转换
private:
    std::size_t val;
};
```

589 可以使用这个类将两个 SmallInt 对象相加，但如果我们试图执行混合模式的算术运算，就将遇到二义性的问题：

```
SmallInt s1, s2;
SmallInt s3 = s1 + s2;    // 使用重载的 operator+
int i = s3 + 0;           // 二义性错误
```

第一条加法语句接受两个 SmallInt 值并执行+运算符的重载版本。第二条加法语句具有二义性：因为我们可以把 0 转换成 SmallInt，然后使用 SmallInt 的+；或者把 s3 转换成 int，然后对于两个 int 执行内置的加法运算。



如果我们对同一个类既提供了转换目标是算术类型的类型转换，也提供了重载的运算符，则将会遇到重载运算符与内置运算符的二义性问题。

14.9.3 节练习

练习 14.52：在下面的加法表达式中分别选用了哪个 operator+？列出候选函数、可行函数及为每个可行函数的实参执行的类型转换：

```
struct LongDouble {
    // 用于演示的成员 operator+；在通常情况下+是个非成员
    LongDouble operator+(const SmallInt&);
    // 其他成员与 14.9.2 节（第 521 页）一致
};

LongDouble operator+(LongDouble&, double);
SmallInt si;
LongDouble ld;
ld = si + ld;
ld = ld + si;
```

练习 14.53：假设我们已经定义了如第 522 页所示的 SmallInt，判断下面的加法表达式是否合法。如果合法，使用了哪个加法运算符？如果不合法，应该怎样修改代码才能使其合法？

```
SmallInt s1;
double d = s1 + 3.14;
```

小结

590

一个重载的运算符必须是某个类的成员或者至少拥有一个类类型的运算对象。重载运算符的运算对象数量、结合律、优先级与对应的用于内置类型的运算符完全一致。当运算符被定义为类的成员时，类对象的隐式 `this` 指针绑定到第一个运算对象。赋值、下标、函数调用和箭头运算符必须作为类的成员。

如果类重载了函数调用运算符 `operator()`，则该类的对象被称作“函数对象”。这样的对象常用在标准函数中。`lambda` 表达式是一种简便的定义函数对象类的方式。

在类中可以定义转换源或转换目的是该类型本身的类型转换，这样的类型转换将自动执行。只接受单独一个实参的非显式构造函数定义了从实参类型到类类型的类型转换；而非显式的类型转换运算符则定义了从类类型到其他类型的转换。

术语表

调用形式 (call signature) 表示一个可调对象的接口。在调用形式中包括返回类型以及一个实参类型列表，该列表在一对圆括号内，实参类型之间以逗号分隔。

类类型转换 (class-type conversion) 包括由构造函数定义的从其他类型到类类型的转换以及由类型转换运算符定义的从类类型到其他类型的转换。只接受单独一个实参的非显式构造函数定义了从实参类型到类类型的转换；而类型转换运算符则定义了从类类型到某个指定类型的转换。

类型转换运算符 (conversion operator) 是类的成员函数，定义了从类类型到其他类型的转换。类型转换运算符必须是它要转换的类的成员，并且通常被定义为常量成员。这类运算符既没有返回类型，也不接受参数。它们返回一个可变为转换运算符类型的值，也就是说，`operator int` 返回一个 `int`，`operator string` 返回一个 `string`，依此类推。

显式的类型转换运算符 (explicit conversion operator) 由关键字 `explicit` 限定的类

型转换运算符。这样的运算符用于条件中的隐式类型转换。

函数对象 (function object) 定义了重载调用运算符的对象。在需要使用函数的地方都能使用函数对象。

函数表 (function table) 形如 `map` 或 `vector` 的容器，容器中所存的值可以被调用。

函数模板 (function template) 能够表示任意可调用类型的标准库模板。

重载的运算符 (overloaded operator) 重定义了某种内置运算符的含义的函数。重载的运算符函数含有关键字 `operator`，之后是要定义的符号。重载的运算符必须含有至少一个类类型的运算对象。重载运算符的优先级、结合律、运算对象数量都与其内置版本一致。

用户定义的类型转换 (user-defined conversion) 类类型转换的同义词。